
Decision Trees

Contact Session 6 — Module 5

*A model you can read like a flowchart. We build it the way you'd play Twenty Questions —
by always asking the question that clears up the most confusion.*

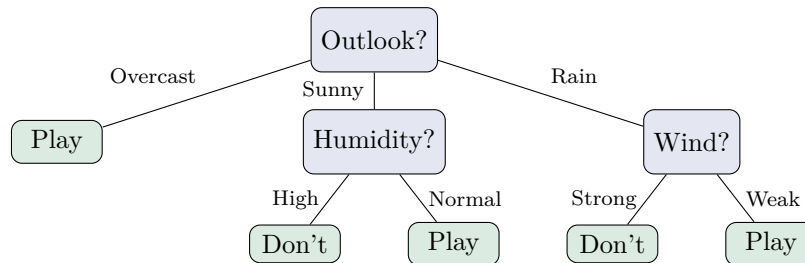
- | | |
|------------------|------------------|
| ■ Intuition | ■ Analogy |
| ■ Worked example | ■ Solved problem |
| ■ Common pitfall | ■ Key takeaways |

Contents

1	The idea: a flowchart the computer writes for you	3
2	Measuring confusion	4
2.1	Where that formula comes from (it's friendlier than it looks)	4
3	Picking the best question	6
4	Watch a tree get built: <i>should we play tennis?</i>	7
5	A trap, and the fix	9
6	Knowing when to stop (overfitting)	10
7	Real-world odds and ends	11
8	The good, the bad, and what comes next	12
9	The whole story on one page	13

1 The idea: a flowchart the computer writes for you

A **decision tree** is just a flowchart of yes/no-ish questions. You start at the top, answer a question about your example, follow the matching branch, answer the next question, and keep going until you reach a final answer at the bottom.



The best part: the computer *builds the flowchart itself* from the data, and you can read the result like plain English (“if it’s overcast, play; if it’s sunny and humid, don’t”). No other model is this easy to explain to a human.

↪ It’s the game of Twenty Questions

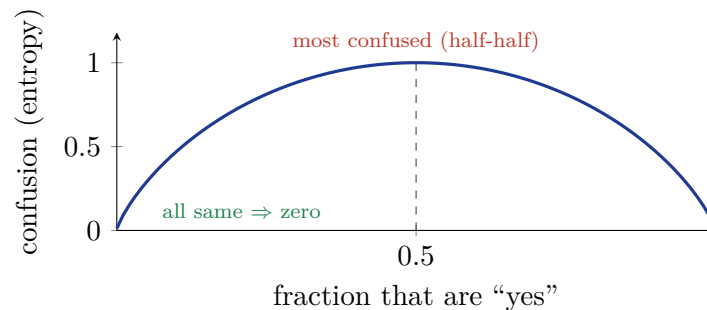
A good Twenty-Questions player never asks at random. They ask the question that splits the remaining possibilities most evenly — “Is it alive?” before “Is it a platypus?” Building a decision tree is exactly this: at each step, pick the question that does the most to clear up the confusion about the answer.

2 Measuring confusion

To pick the *best* question, we first need to measure how “mixed up” a group of examples is. Imagine a basket of examples, each labelled yes or no.

- All yes (or all no)? **Zero confusion** — you already know the answer, no question needed.
- Half yes, half no? **Maximum confusion** — it’s a coin flip.
- Mostly yes, a few no? **Somewhere in between.**

This “amount of confusion” has a name: **entropy**. You don’t need to love the formula to use the idea — just remember the picture:



When you do want the number, it’s

$$\text{confusion} = -p_{\text{yes}} \log_2 p_{\text{yes}} - p_{\text{no}} \log_2 p_{\text{no}},$$

which is just a tidy way to get “0 when pure, 1 when perfectly mixed.” Read p_{yes} as “the fraction that are yes.”

★ Confusion = how many yes/no questions to be sure

Entropy is really “how many yes/no questions would I need, on average, to pin down a random example’s label?” If the basket is all-yes, you need zero — you already know. If it’s a perfect coin-flip, you need one. That’s why a 50/50 basket scores exactly 1.

2.1 Where that formula comes from (it’s friendlier than it looks)

The formula is really two simple ideas glued together.

Idea 1: surprise. How surprised should you be by something that had probability p ? If it was almost certain (p near 1) and it happens — no surprise. If it was rare (p tiny) and it happens — big surprise. The quantity that captures this is $\log_2 \frac{1}{p}$, which is the same as $-\log_2 p$. It is 0 when $p = 1$ (certain, no surprise) and grows as p gets smaller.

Idea 2: average it. Entropy is just the *average* surprise over the labels — each outcome’s surprise, weighted by how often it happens:

$$H = \underbrace{p_{\text{yes}}}_{\text{how often}} \times \underbrace{\log_2 \frac{1}{p_{\text{yes}}}}_{\text{its surprise}} + p_{\text{no}} \times \log_2 \frac{1}{p_{\text{no}}} = -\sum p \log_2 p.$$

That’s the whole thing: *how surprised will I be, on average, by a random label?* The minus sign out front is only there because we flipped $\log \frac{1}{p}$ into $-\log p$ — nothing deep.

Why base-2? It makes the answer come out in **bits**, i.e. yes/no questions. Two quick checks against the formula:

$$50/50: -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = -\frac{1}{2}(-1) - \frac{1}{2}(-1) = 1 \text{ bit}, \quad \text{all-yes: } \log_2 1 = 0 \Rightarrow H = 0.$$

The curve you saw above is simply this formula drawn out.

★ A cousin you’ll meet: the Gini score

Some trees measure mess with **Gini** instead of entropy:

$$\text{Gini} = 1 - \sum p^2 = 1 - (p_{\text{yes}}^2 + p_{\text{no}}^2).$$

It has a cute meaning: *the chance that two random grabs from the basket pull out two **different** labels*. An all-same basket $\Rightarrow$ you always grab the same label $\Rightarrow$ Gini = 0. A half-half basket $\Rightarrow 1 - (0.25 + 0.25) = 0.5$, the messiest. Same story as entropy (0 means pure, bigger means messier), just quicker to compute — which is why the popular CART trees prefer it.

3 Picking the best question

Now the key move. Take a question (say “Is it sunny?”). It splits the basket into smaller baskets (sunny days, overcast days, rainy days). Measure the confusion *before* the split, and the average confusion *after*. The drop is how much the question helped:

$$\text{information gain} = (\text{confusion before}) - (\text{average confusion after}).$$

Writing the same thing in symbols, if question A splits the basket S into smaller baskets S_v :

$$\text{Gain}(S, A) = \underbrace{H(S)}_{\text{mess before}} - \underbrace{\sum_v \frac{|S_v|}{|S|} H(S_v)}_{\text{average mess after}}.$$

The only fiddly part is those weights $\frac{|S_v|}{|S|}$: each little basket counts in proportion to *how many examples fell into it* — a pile of 8 days matters more than a pile of 2. (Forgetting that weighting is the number-one slip people make.)

The rule is simply: at each step, ask the question with the biggest drop in confusion. That greedy little rule, repeated, builds the whole tree. (The classic version of this is called *ID3*.)

↪ A sieve that sorts marbles

Each possible question is a sieve. Pour your mixed marbles through and look at the piles it makes. A *great* sieve leaves each pile almost all-one-colour (low confusion, big drop). A *useless* sieve leaves every pile just as mixed as before (no drop). Pick the sieve that sorts best, then sort each pile again with the next-best sieve.

4 Watch a tree get built: *should we play tennis?*

Here’s the famous little dataset — 14 days, each described by the weather, and whether we played. Overall: **9 play days, 5 don’t**.

Day	Outlook	Temp	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

The starting confusion (9 play, 5 don’t) comes straight from the formula:

$$H = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} \approx 0.94 \text{ bits},$$

which is high (close to 1) because the basket is quite mixed. Now we test each question and see which clears up the most confusion.

Σ Does “Is it windy?” help much?

Split by Wind. Weak-wind days: 8 of them, 6 play / 2 don’t (fairly lopsided, confusion ≈ 0.81). Strong-wind days: 6 of them, 3 play / 3 don’t (a perfect coin-flip, confusion = 1.0). Average confusion after the split:

$$\frac{8}{14}(0.81) + \frac{6}{14}(1.0) \approx 0.89.$$

Drop = $0.94 - 0.89 = \mathbf{0.05}$. Barely helps. (*Note: weight each little basket by how many days are in it — forgetting that weighting is the #1 mistake here.*)

Σ Does “Is it humid?” help more?

Split by Humidity. High: 7 days, 3 play / 4 don’t (confusion ≈ 0.99). Normal: 7 days, 6 play / 1 don’t (much cleaner, confusion ≈ 0.59). Average after:

$$\frac{7}{14}(0.99) + \frac{7}{14}(0.59) \approx 0.79.$$

Drop = $0.94 - 0.79 = \mathbf{0.15}$. Three times better than Wind.

Doing this for all four questions gives drops of: **Outlook** 0.25, Humidity 0.15, Wind 0.05, Temperature 0.03. **Outlook wins**, so it becomes the top question. Then we repeat *inside* each branch:

- **Overcast** days — all 4 are “play.” Zero confusion, so we stop: a leaf that says *Play*.
- **Sunny** days — splitting by Humidity sorts them perfectly (High → Don’t, Normal → Play).
- **Rain** days — splitting by Wind sorts them perfectly (Strong → Don’t, Weak → Play).

And that’s the exact tree on page 1 — built entirely by “ask the question that clears the most confusion.”

5 A trap, and the fix

The “biggest drop” rule has one blind spot: it *loves* questions with tons of answers. Imagine using “What is the Day number?” as a question — it gives each day its own tiny basket of one, every basket perfectly pure, an enormous drop. But it’s useless: it has just memorised the days and learned nothing about *weather*.

★ Count the children, not just their purity

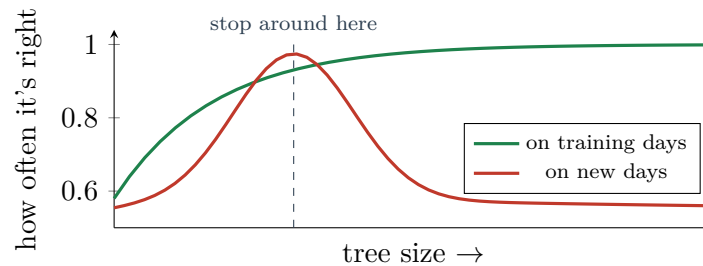
The plain rule asks “how pure are the resulting piles?” but ignores “how many piles did I make?” A question that shatters the data into hundreds of one-item piles looks amazing but is really just memorising. The fix, called **gain ratio**, divides the credit by how badly the question fragmented the data:

$$\text{Gain Ratio} = \frac{\text{Gain}}{\text{Split Info}}, \quad \text{Split Info} = - \sum_v \frac{|S_v|}{|S|} \log_2 \frac{|S_v|}{|S|}.$$

Notice that “Split Info” is just our *entropy formula again* — but applied to the *sizes* of the piles, not their labels. A question that makes many tiny piles has a big Split Info, which divides its gain back down to a sensible size. So “which day?” (365 tiny piles) gets heavily discounted, while “is it sunny?” (3 fat piles) barely loses anything.

6 Knowing when to stop (overfitting)

If we keep splitting until every leaf is pure, the tree memorises the training days perfectly — including their flukes — and then stumbles on new days. As the tree grows, its score on training data keeps climbing, but its score on *fresh* data climbs, peaks, and then *falls*. That falling part is over-fitting.



Two cures, both common sense: **stop early** (don't split a basket that's already tiny or barely improving), or **grow the full tree, then prune it back** (snip off branches that don't help on a held-aside check set). Pruning usually works a little better.

★ Occam's razor, in bits

There's a tidy way to say “don't over-grow”: prefer the tree that lets you *describe the data most briefly*. A bigger tree explains the training days with fewer mistakes, but the tree itself is longer to write down. The sweet spot is the tree with the shortest *total* story = (tree description) + (its leftover mistakes). Small enough to be simple, big enough to be right.

7 Real-world odds and ends

Numbers, not categories. For a fact like Temperature = 72, the tree invents a yes/no question on the fly: “is Temperature below some cut-off?” It tries the promising cut-offs and keeps the best one.

Missing entries. If a day is missing its Humidity, you don’t throw the day away. You can fill in the most common value, or cleverly send the day *part-way down each branch* in proportion to how the other days split. Trees handle gaps gracefully so you don’t lose data.

8 The good, the bad, and what comes next

Great about trees	Not so great
you can read them like rules	jumpy — a little new data can re-shape the whole tree
no need to scale features	greedy — may miss the very best tree
handle mixed and missing data	fences are only straight cuts (no slanted borders)
fast to use	overfit if you don't prune

That “jumpy” weakness is exactly why people rarely use *one* tree. In Module 9 we’ll grow *many* trees and average them (**Random Forests**) — which keeps the flexibility but irons out the jumpiness, and is one of the most reliable methods around.

9 The whole story on one page

★ Key Takeaways

- A decision tree is a **flowchart of questions** the computer builds itself; each path reads like a plain rule.
- Measure how mixed-up a basket is with **confusion (entropy)**: 0 when all-same, 1 when perfectly half-and-half.
- At each step, ask the question with the biggest **drop in confusion** (information gain). Repeat inside each branch.
- Watch out: the rule over-rewards questions with *many* answers — fix with **gain ratio**.
- Full trees **over-fit**; cure by **stopping early** or **pruning** back on a check set. (Briefest-total-description is the principle behind it.)
- Numbers become “below a cut-off?” questions; missing entries are filled or split gracefully.
- One tree is jumpy — averaging **many** trees (Random Forests, Module 9) fixes that.

Where this comes from: Mitchell, *Machine Learning*, Ch. 3; Tan, Steinbach & Kumar, Ch. 3; Quinlan (ID3, C4.5). Course syllabus Module 5 (BITS Pilani WILP).